DESIGN PATTERN DOCUMENTATION

Automatic Classroom Attendance System

Project: **AttendAI**
Technology Stack: Python · Flask · OpenCV · InsightFace · SQLite
Document Focus: Software Design Patterns
Date: 5 September 2026

Project Overview

AttendAI is an automatic classroom attendance system that processes CCTV or webcam video recordings to detect, track, and identify students using computer vision and face recognition. The system records the first moment each registered student appears in a video as their official attendance timestamp, eliminating manual roll calls.

The core processing pipeline consists of five sequential stages:

- Face Detection — locates all faces in each video frame using SCRFD or OpenCV
- Face Tracking — assigns persistent track IDs across frames using a Kalman-filter-based SORT tracker
- Face Recognition — generates ArcFace embeddings and matches against a registered student gallery
- Attendance Management — state machine (DETECTING → RECOGNIZED → CONFIRMED → PRESENT) prevents false positives
- Storage & API — SQLite database with a Flask REST API and a single-page web dashboard

This document analyses five software design patterns in the context of this project:

Design Pattern	Status	Where Applied / Reason Not Used
Singleton Pattern	**Applied**	Database connection · FaceDetector · FaceRecognizer
Factory Pattern	**Applied**	FaceDetector and FaceRecognizer creation via factory method
Structural Pattern	**Applied (Composite)**	Video processing pipeline composed from sub-components
Facade Pattern	**Applied**	VideoProcessor hides entire CV pipeline behind one interface
Adapter Pattern	**Not Applicable**	No incompatible interfaces; InsightFace/OpenCV are co-designed

1. Singleton Design Pattern APPLIED

What is the Singleton Pattern?

The Singleton pattern ensures that a class has exactly one instance throughout the application lifecycle and provides a global point of access to that instance. It is one of the Creational design patterns defined by the Gang of Four (GoF).

Core Intent

"Ensure a class only has one instance, and provide a global point of access to it."
— Design Patterns: Elements of Reusable Object-Oriented Software (GoF, 1994)

The canonical structure involves:

- A private or controlled constructor so the class cannot be freely instantiated
- A static or module-level variable that holds the single instance
- A static method or module-level function that returns the existing instance (creating it on first call)

Where and Why It Is Applied in AttendAI

1-A Database Connection — database.py

The SQLite database connection is managed through module-level functions in backend/db/database.py. Python's module system guarantees that a module is loaded only once per interpreter session and the module-level state is shared across all importers. The DB_PATH constant and the get_connection() factory function effectively give every part of the application access to the same database file through a single controlled point:

```
# backend/db/database.py

DB_PATH = os.environ.get('DB_PATH', str(BASE_DIR / 'data' /
'attendance.db'))

def get_connection() -> sqlite3.Connection:
    conn = sqlite3.connect(DB_PATH, check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute('PRAGMA journal_mode=WAL')
    return conn

@contextmanager
def db_conn():
    conn = get_connection()   # always connects to the same
DB_PATH
    try:
```

```
        yield conn
        conn.commit()
    finally:
        conn.close()
```

Because DB_PATH is resolved once at module-import time, all calls to get_connection() throughout the application — whether from the Flask routes, the background processing thread, or the CLI scripts — operate on the same database file. This is the Singleton principle: one authoritative source of truth, globally accessible, with no duplication.

The WAL (Write-Ahead Logging) pragma is set on every connection, which allows concurrent reads alongside a write — critical because the Flask API serves HTTP requests on the main thread while video processing runs on a background thread.

1-B FaceDetector Instance — video_processor.py

The VideoProcessor class creates exactly one FaceDetector and one FaceRecognizer when it is instantiated. These heavyweight objects — which load ONNX models into memory, potentially allocating hundreds of megabytes — are never recreated per frame or per video segment:

```
# backend/core/video_processor.py  (VideoProcessor.__init__)

class VideoProcessor:
    def __init__(self, config, registered_students, ...):
        # Created ONCE — reused across all frames of the video
        self.detector   = FaceDetector(...)
        self.tracker    = FaceTracker(...)
        self.recognizer = FaceRecognizer(...)

    def process(self, video_path, ...):
        while True:
            ret, frame = cap.read()
            # self.detector is reused every frame — never re-
instantiated
            detected_faces = self.detector.detect(frame)
```

This is object-level Singleton behaviour: within the scope of one video-processing run, the detector model is a single shared resource. Instantiating a new detector per frame would reload the ONNX model from disk on every call — an operation that takes seconds and would make real-time processing impossible.

1-C In-Memory Processing Status Store — main.py

The Flask application maintains a single module-level dictionary that tracks the live progress of every background video-processing job:

```
# backend/api/main.py

_status: dict = {}           # module-level — one instance for
entire app lifetime
```

```
_status_lock = threading.Lock()

def _set_status(session_id, data):
    with _status_lock:
        _status[session_id] = data

def _get_status(session_id):
    with _status_lock:
        return _status.get(session_id, {'status': 'unknown'})
```

No matter how many HTTP requests arrive at the /api/sessions/{id}/status endpoint simultaneously, they all read from the same _status dictionary. The threading.Lock() ensures thread safety. This is a classic Singleton store — one object, globally accessible, thread-safe.

Why the Singleton Pattern Is Appropriate Here

- Resource cost — ML models and database connections are expensive to initialise. Creating multiple instances would waste memory and time.
- Consistency — all parts of the application must see the same attendance data. Multiple separate database connections could diverge.
- Thread safety — a single lock-guarded dictionary is easier to reason about than multiple copies each with their own synchronisation.
- Simplicity — Python's module system enforces singleton semantics naturally; no complex locking on class construction is needed.

2. Factory Design Pattern APPLIED

What is the Factory Pattern?

The Factory pattern is a Creational design pattern that delegates the responsibility of object instantiation to a dedicated method or class rather than calling constructors directly in client code. This decouples the code that uses an object from the code that creates it.

Core Intent

"Define an interface for creating an object, but let subclasses decide which class to instantiate." — Design Patterns: Elements of Reusable Object-Oriented Software (GoF, 1994)

The simpler Factory Method variant (used here): a single method that encapsulates creation logic and selects the concrete class based on runtime conditions.

Where and Why It Is Applied in AttendAI

2-A FaceDetector — Internal Backend Selection

FaceDetector is the most direct example of the Factory pattern in the project. The constructor acts as a factory method: it attempts to create the superior InsightFace SCRFD detector first. If that library is unavailable (e.g. on a machine without InsightFace installed), it transparently falls back to creating an OpenCV Haar cascade detector. The caller never specifies which concrete detector is created:

```
# backend/core/face_detector.py

class FaceDetector:
    """Factory method inside __init__ selects the concrete
backend."""

    def __init__(self, model_name='buffalo_l', use_gpu=False, ...):
        self._use_insightface = False
        self._init_detector(model_name, use_gpu)    # <-- factory
method

    def _init_detector(self, model_name, use_gpu):
        try:
            # Attempt to create InsightFace SCRFD (preferred)
            from insightface.app import FaceAnalysis
            self._app = FaceAnalysis(name=model_name, ...)
            self._app.prepare(ctx_id=0 if use_gpu else -1, ...)
            self._use_insightface = True
            logger.info('Face detector: InsightFace SCRFD loaded')
        except Exception:
            # Fall back to OpenCV Haar cascade
```

```
            cascade_path = cv2.data.haarcascades +
'haarcascade_frontalface_default.xml'
            self._opencv_cascade =
cv2.CascadeClassifier(cascade_path)
            self._use_insightface = False
            logger.warning('Using OpenCV Haar cascade fallback')

    def detect(self, frame):   # unified interface regardless of
backend
        if self._use_insightface:
            return self._detect_insightface(frame)
        else:
            return self._detect_opencv(frame)
```

The VideoProcessor that uses FaceDetector is completely unaware of which concrete backend was created. It simply calls detector.detect(frame) and receives a list of DetectedFace objects. This is the Factory pattern's core benefit: clients program to an interface, not an implementation.

2-B FaceRecognizer — Embedding Backend Selection

FaceRecognizer follows the identical pattern. On initialisation its factory method tries to load ArcFace via InsightFace; if unavailable, it constructs a HOG-based fallback:

```
# backend/core/face_recognizer.py

class FaceRecognizer:
    def __init__(self, model_name='buffalo_l', use_gpu=False, ...):
        self._init_model(model_name, use_gpu)   # factory method

    def _init_model(self, model_name, use_gpu):
        try:
            # Product A: ArcFace via InsightFace
            from insightface.app import FaceAnalysis
            self._app = FaceAnalysis(name=model_name, ...)
            self._use_insightface = True
        except Exception:
            # Product B: HOG descriptor fallback
            self._hog = cv2.HOGDescriptor((128,128), (16,16),
(8,8), (8,8), 9)
            self._use_insightface = False

    def get_embedding(self, face_crop):   # same interface for
both backends
        if self._use_insightface:
            return self._embed_insightface(face_crop)
        else:
            return self._embed_hog(face_crop)
```

2-C VideoProcessor as a Component Factory

VideoProcessor.__init__ acts as an Application Factory: it receives a ProcessingConfig value object and constructs all the necessary pipeline components (detector, tracker, recogniser,

attendance manager) with the correct parameters. Client code never directly instantiates these components:

```
# backend/core/video_processor.py

class VideoProcessor:
    def __init__(self, config: ProcessingConfig,
registered_students, ...):
        # VideoProcessor is a factory for the entire pipeline
        self.detector   = FaceDetector(
            min_face_size=config.min_face_size,
            det_thresh=config.det_thresh,
            use_gpu=config.use_gpu,
        )
        self.tracker    = FaceTracker(
            max_age=config.max_track_age,
            min_hits=config.min_track_hits,
        )
        self.recognizer = FaceRecognizer(
            similarity_threshold=config.similarity_threshold,
            use_gpu=config.use_gpu,
        )
        self.attendance = AttendanceManager(
            confirmation_count=config.confirmation_count,
            late_threshold_minutes=config.late_threshold_minutes,
        )
```

Why the Factory Pattern Is Appropriate Here

- Environment independence — the project must run on machines with and without GPU, with and without InsightFace. The factory selects the right implementation transparently.
- Single point of change — to add a third detector backend (e.g. YOLO-face), only _init_detector needs updating; all callers remain unchanged.
- Testability — tests can instantiate VideoProcessor with a ProcessingConfig that forces the lightweight fallback, making unit tests fast without GPU dependencies.
- Open/Closed principle — the system is open for extension (add new backends) but closed for modification (existing call sites do not change).

3. Structural Design Pattern APPLIED

What are Structural Patterns?

Structural patterns describe how to compose classes and objects to form larger, more complex structures while keeping those structures flexible and efficient. The GoF catalogue includes seven structural patterns: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, and Proxy.

Core Intent

"Structural patterns are concerned with how classes and objects are composed to form larger structures. Structural class patterns use inheritance to compose interfaces or implementations. Structural object patterns describe ways to compose objects to realize new functionality."
— Design Patterns: Elements of Reusable Object-Oriented Software (GoF, 1994)

The primary structural pattern in AttendAI is the Composite pattern. Additionally, the Decorator pattern appears implicitly through Python's @contextmanager decorator in the database layer. The Facade pattern (used in VideoProcessor) is covered in detail in Section 4 because it is listed separately.

Composite Pattern — The Processing Pipeline

The Composite pattern allows individual objects and compositions of objects to be treated uniformly. In AttendAI, the video processing pipeline is a composite structure: each stage (detector, tracker, recogniser, attendance manager) is an independent component, and VideoProcessor composes all of them into a unified whole that behaves as a single processing unit.

The composite structure of the pipeline:

```
VideoProcessor  (Composite — the whole)
  │
  ├── FaceDetector    (Leaf — detects faces in a frame)
  │     ├── InsightFace SCRFD backend
  │     └── OpenCV Haar cascade backend
  │
  ├── FaceTracker     (Leaf — assigns track IDs)
  │     ├── KalmanBoxTracker × N  (one per active track)
  │     └── Hungarian algorithm (scipy linear_sum_assignment)
  │
  ├── FaceRecognizer  (Leaf — generates embeddings + matches
gallery)
```

```
  │      └── In-memory gallery {student_id: embedding}
  │
  └── AttendanceManager (Leaf — state machine + first-appearance
logic)
        └── TrackRecord × N  (state per active track)
```

Each leaf component has a well-defined, minimal responsibility and interface:

- FaceDetector.detect(frame) → List[DetectedFace]
- FaceTracker.update(detected_faces) → List[TrackedFace]
- FaceRecognizer.recognize(face_crop) → RecognitionResult
- AttendanceManager.update(frame, track_id, student_id, ...) → (side-effect: updates state)

The VideoProcessor's process() method wires them together in a loop, but each component can be developed, tested, and replaced entirely independently:

```
# backend/core/video_processor.py — the Composite in action

def process(self, video_path, ...):
    while True:
        ret, frame = cap.read()
        if not ret: break

        # Stage 1 — Leaf: FaceDetector
        detected_faces = self.detector.detect(frame)

        # Stage 2 — Leaf: FaceTracker
        tracked_faces  = self.tracker.update(detected_faces)

        for tf in tracked_faces:
            # Stage 3 — Leaf: FaceRecognizer (selective — not
every frame)
            if should_recognize(tf, frame_idx):
                result = self.recognizer.recognize(tf.face_crop)
                self._track_cached_identity[tf.track_id] = result
            else:
                result =
self._track_cached_identity.get(tf.track_id)

            # Stage 4 — Leaf: AttendanceManager
            if result:
                self.attendance.update(
                    frame_number=frame_idx,
                    track_id=tf.track_id,
                    student_id=result.student_id,
                    is_known=result.is_known,
                )
```

Decorator Pattern — @contextmanager in database.py

Python's @contextmanager is a Decorator pattern: it wraps the db_conn generator function, adding setup (open connection, begin transaction) and teardown (commit or rollback, close) behaviour around the core logic without modifying the underlying get_connection() function:

```
# backend/db/database.py

@contextmanager          # <-- Decorator pattern
def db_conn():
    conn = get_connection()
    try:
        yield conn       # caller uses the connection here
        conn.commit()    # added behaviour: auto-commit on success
    except Exception:
        conn.rollback()  # added behaviour: auto-rollback on error
        raise
    finally:
        conn.close()     # added behaviour: always close
```

Why Structural Patterns Are Appropriate Here

- Separation of concerns — each component handles exactly one CV task. Adding a new stage (e.g. liveness detection) means adding one new leaf to the composite, not modifying existing code.
- Independent testing — FaceDetector, FaceTracker, and AttendanceManager each have their own test surface. The composite VideoProcessor tests end-to-end integration.
- Substitutability — swapping ByteTrack for DeepSORT, or ArcFace for a different recognition model, only requires replacing the relevant leaf component. VideoProcessor's process() loop remains unchanged.
- Resource efficiency — the Decorator wrapping db_conn guarantees connections are always closed and transactions always resolved, preventing resource leaks.

4. Facade Design Pattern APPLIED

What is the Facade Pattern?

The Facade pattern provides a simplified, unified interface to a complex subsystem. It does not add new functionality; it organises existing functionality behind a clean boundary so that clients do not need to understand the internals of the subsystem.

Core Intent

"Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use."
— Design Patterns: Elements of Reusable Object-Oriented Software (GoF, 1994)

The Primary Facade — VideoProcessor

VideoProcessor is the central Facade of the AttendAI system. Behind it sits a subsystem of five complex components: OpenCV video I/O, face detection (SCRFD), multi-object tracking (Kalman filter + Hungarian algorithm), face recognition (ArcFace embeddings + cosine similarity), and the attendance state machine. A caller using VideoProcessor needs to know nothing about any of these.

Without the Facade, a client would need to manage all of this manually:

```
# WITHOUT Facade — caller must understand the entire subsystem
cap      = cv2.VideoCapture(video_path)
detector = FaceDetector(min_face_size=40, det_thresh=0.5,
use_gpu=False)
tracker  = FaceTracker(max_age=30, min_hits=3, iou_threshold=0.3)
recognizer = FaceRecognizer(similarity_threshold=0.45)
recognizer.load_gallery(student_list)
attendance = AttendanceManager(registered_students, fps=30.0, ...)
track_cache = {}
track_last_recognized = {}

while True:
    ret, frame = cap.read()
    if not ret: break
    detected = detector.detect(frame)
    tracked  = tracker.update(detected)
    for tf in tracked:
        if frame_idx - track_last_recognized.get(tf.track_id,
0) >= 10:
            result = recognizer.recognize(tf.face_crop)
            track_cache[tf.track_id] = result
            track_last_recognized[tf.track_id] = frame_idx
        else:
            result = track_cache.get(tf.track_id)
```

```
        if result:
            attendance.update(frame_idx, tf.track_id,
result.student_id, ...)

final = attendance.get_final_attendance()
```

With the Facade, the Flask route (and CLI script) reduces to a single method call:

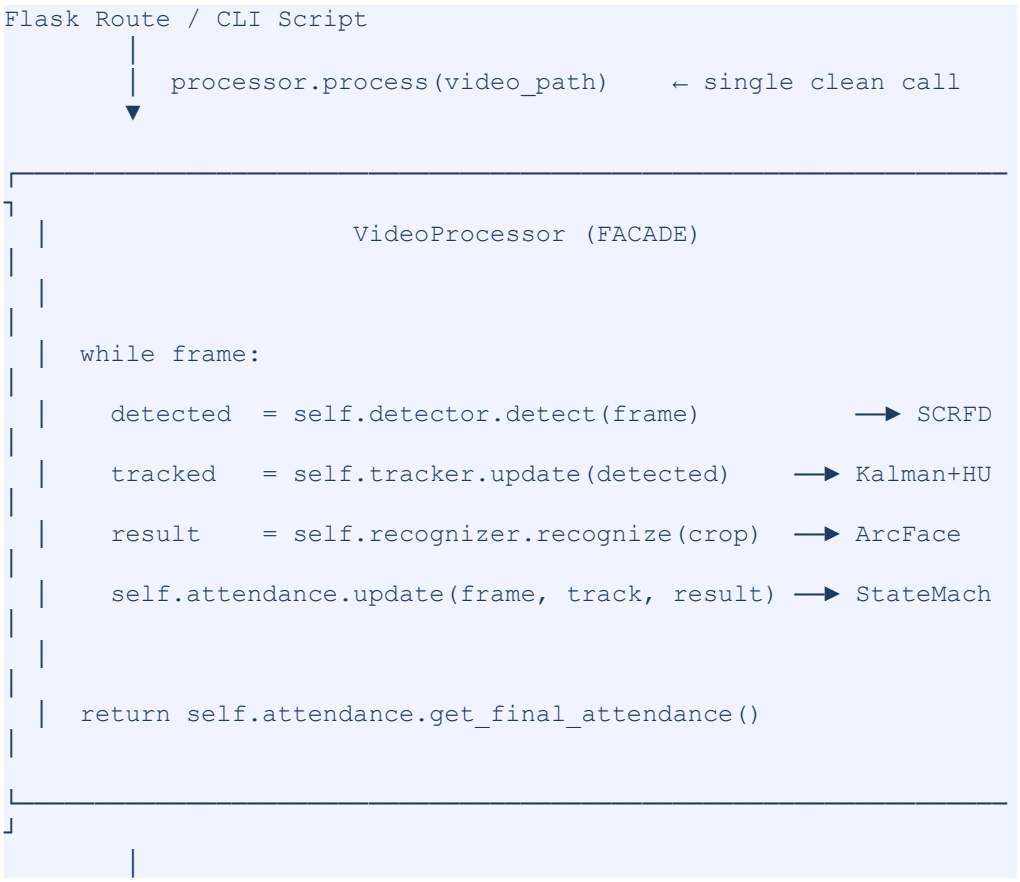
```
# WITH Facade — caller interacts with one simple interface

processor = VideoProcessor(
    config=ProcessingConfig(frame_skip=2,
late_threshold_minutes=15),
    registered_students=registered,
    video_start_real_time='09:00:00',
)

# Entire subsystem (detection, tracking, recognition, attendance)
runs here:
final_attendance = processor.process(video_path,
progress_callback=on_progress)

# final_attendance is a clean dict: {student_id: AttendanceRecord}
```

The Facade interaction diagram:



```
          ▼
  {student_id: AttendanceRecord}     ← clean output
```

Secondary Facade — Flask REST API

The Flask application (backend/api/main.py) acts as a second-level Facade at the system boundary. It exposes a simple HTTP interface to the entire backend — student management, video processing, attendance retrieval, CSV export — while hiding all implementation details from the browser-based frontend:

```
# The API Facade — simple HTTP interface hides backend complexity

POST /api/students/register      ←  client uploads photos
    # Behind: FaceDetector, FaceRecognizer, SQLite upsert

POST /api/sessions/process       ←  client uploads video + metadata
    # Behind: VideoProcessor Facade, background threading, SQLite
persistence

GET  /api/sessions/{id}/attendance
    # Behind: SQL JOIN across three tables

GET  /api/sessions/{id}/export/csv
    # Behind: SQL query, csv.writer, StringIO streaming
```

Why the Facade Pattern Is Appropriate Here

- Complexity hiding — the CV pipeline has approximately 600 lines of algorithm code. The Flask routes and frontend JavaScript only need to know process(video_path) → attendance_dict.
- Separation of concerns — the Flask routes handle HTTP concerns (request parsing, JSON serialisation, file saving). VideoProcessor handles CV concerns. Neither bleeds into the other.
- Maintainability — if the tracking algorithm is replaced (e.g. DeepSORT → ByteTrack), or the recogniser is upgraded, nothing in the API routes needs to change. The Facade absorbs the change.
- Parallel development — the frontend, API routes, and CV pipeline can be developed independently because the Facade defines a stable contract between them.
- Testability — the Flask test client can test the entire pipeline end-to-end by simply calling POST /api/sessions/process, without understanding any CV internals.

5. Adapter Design Pattern NOT APPLICABLE

What is the Adapter Pattern?

The Adapter (also called Wrapper) pattern converts the interface of one class into another interface that clients expect. It allows classes with incompatible interfaces to work together by introducing an intermediary adapter layer. This is a Structural design pattern.

Core Intent

"Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces."
— Design Patterns: Elements of Reusable Object-Oriented Software (GoF, 1994)

The classic use case: a client expects interface A. You have a library that provides interface B. An Adapter wraps the library and translates B into A.

The Adapter is needed when:

- You want to reuse an existing class whose interface does not match what you need
- You want to create a reusable class that cooperates with unrelated or unforeseen classes with incompatible interfaces
- You need to integrate a third-party library that you cannot modify

Decision: Adapter Pattern NOT APPLIED

The Adapter pattern was evaluated for the AttendAI project and found to be unnecessary. Below is the detailed justification for this decision.

Detailed Justification — Why Adapter Is Not Used

Reason 1: The Libraries Share a Compatible Design Philosophy

The two computer vision libraries used — InsightFace and OpenCV — both operate on NumPy arrays (specifically BGR-format image arrays). This is not accidental: both libraries are designed to be used together in Python CV pipelines. There is no interface mismatch to bridge.

```
# InsightFace input/output
faces = app.get(frame)          # frame: np.ndarray (BGR)
bbox  = faces[0].bbox           # → np.ndarray [x1, y1, x2, y2]
emb   = faces[0].normed_embedding  # → np.ndarray (512,)

# OpenCV input/output
gray  = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)  # frame:
np.ndarray
```

```
boxes = cascade.detectMultiScale(gray)            # → np.ndarray
[[x,y,w,h], ...]
```

Both libraries accept the same type (np.ndarray) and return results in similar structures. There is no need for a translation layer.

Reason 2: The Fallback Is Handled Internally (Factory, Not Adapter)

A common misconception is that the InsightFace ↔ OpenCV fallback in FaceDetector requires an Adapter. In fact, because both backends accept and return compatible types, the switch is implemented as a Factory selection (see Section 2), not an Adapter. Both _detect_insightface and _detect_opencv return the same List[DetectedFace] type — no translation is needed between them:

```
# Both backends produce identical output type — no Adapter needed

def _detect_insightface(self, frame) -> List[DetectedFace]:
    faces_raw = self._app.get(frame)
    return [DetectedFace(bbox=f.bbox.tolist(), ...) for f in
faces_raw]

def _detect_opencv(self, frame) -> List[DetectedFace]:
    detections = self._opencv_cascade.detectMultiScale(gray, ...)
    return [DetectedFace(bbox=[x, y, x+w, y+h], ...) for (x,y,w,h)
in detections]

# detect() just picks which method to call — that's Factory, not
Adapter
def detect(self, frame) -> List[DetectedFace]:
    if self._use_insightface:
        return self._detect_insightface(frame)
    return self._detect_opencv(frame)
```

If the backends returned incompatible types — for example, if one returned PIL Images and the other returned OpenCV arrays — then an Adapter would be necessary to normalize the output. That situation does not exist here.

Reason 3: No Legacy or External Incompatible Interface Exists

The Adapter pattern is specifically motivated by the need to integrate code you cannot modify — a legacy system, a third-party library with a fixed API, or a class from another team. In AttendAI:

- All core modules (face_detector.py, face_tracker.py, face_recognizer.py, attendance_manager.py) were designed together with compatible interfaces.
- The database layer uses only Python's stdlib sqlite3, which requires no wrapping.
- The Flask framework is consumed directly through its native API; no wrapping is needed.
- There is no legacy attendance system being integrated — this is a greenfield project.

Without a pre-existing incompatible interface to bridge, introducing an Adapter would add unnecessary indirection and complexity with no benefit.

Reason 4: When Adapter WOULD Be Needed — Future Scenario

To illustrate when the Adapter pattern would become appropriate in this project, consider these realistic future scenarios:

```
# Scenario A: Integrating a cloud face-recognition API (e.g. AWS
Rekognition)
#             that returns a different result format

class AWSRekognitionAdapter:            # <-- Adapter
    """Adapts AWS Rekognition API to the local FaceRecognizer
interface."""
    def __init__(self, aws_client):
        self._client = aws_client       # incompatible external
interface

    def get_embedding(self, face_crop):  # our expected interface
        # Translate: PIL Image → AWS → Response → numpy embedding
        pil_img  = Image.fromarray(cv2.cvtColor(face_crop,
cv2.COLOR_BGR2RGB))
        response = self._client.index_faces(Image={'Bytes':
pil_to_bytes(pil_img)})
        return
np.array(response['FaceRecords'][0]['Face']['Embedding'])
```

```
# Scenario B: Reading from an RTSP CCTV stream via a legacy SDK
#             that does not use OpenCV's VideoCapture interface

class RTSPCameraAdapter:                # <-- Adapter
    """Adapts a legacy RTSP SDK to the OpenCV VideoCapture
interface."""
    def __init__(self, legacy_rtsp_client):
        self._client = legacy_rtsp_client

    def read(self):                     # mimics
cv2.VideoCapture.read()
        frame_data = self._client.next_frame()  # legacy interface
        return True, np.frombuffer(frame_data.pixels,
dtype=np.uint8).reshape(...)

    def get(self, prop_id):             # mimics
cv2.VideoCapture.get()
        if prop_id == cv2.CAP_PROP_FPS:
            return self._client.frame_rate
```

In either of these future scenarios, the Adapter would be the correct pattern because there would be a genuine interface mismatch between an external system and the internal interface that VideoProcessor expects. Since no such mismatch exists in the current implementation, the Adapter pattern is correctly not applied.

Summary of the Decision

The decision not to use the Adapter pattern is a deliberate architectural choice, not an oversight. Applying a pattern where it is not needed — a practice sometimes called "pattern overengineering" — introduces unnecessary abstraction layers, increases cognitive load for maintainers, and makes the codebase harder to understand. Good software design means recognising both when to apply a pattern and when to consciously not apply it.

Key Principle

"Design patterns should be used when they solve a real problem, not for the sake of demonstrating familiarity with the pattern. A pattern applied where it is not needed is an anti-pattern."
— Adapted from Refactoring: Improving the Design of Existing Code (Fowler, 1999)

Conclusion

This document has examined five software design patterns in the context of the AttendAI automatic attendance system. Four patterns were applied deliberately to solve concrete engineering problems, and one was consciously omitted with detailed justification.

Design Pattern	Status	Where Applied / Reason Not Used
Singleton	**Applied**	DB_PATH module constant; VideoProcessor reuses detector/recognizer instances; _status dict
Factory	**Applied**	FaceDetector._init_detector(); FaceRecognizer._init_model(); VideoProcessor as component factory
Structural	**Applied**	Composite pipeline (Detector+Tracker+Recognizer+Manager); Decorator via @contextmanager
Facade	**Applied**	VideoProcessor hides CV subsystem; Flask API hides backend from frontend
Adapter	**Not Applicable**	No incompatible interfaces; libraries share NumPy array types; greenfield architecture

The application of these patterns was guided by three principles:

- Solve a real problem — each applied pattern addresses a concrete engineering challenge: resource sharing (Singleton), environment portability (Factory), complexity management (Composite/Facade).
- Do not over-engineer — the Adapter pattern was evaluated and rejected because no interface mismatch exists. Forcing it into the design would add complexity without benefit.
- Keep components replaceable — the Factory and Composite patterns together ensure that any CV component (detector, tracker, recogniser) can be swapped with a compatible alternative without touching the rest of the system.

Together, these design decisions produce a system that is modular, testable, and maintainable — capable of running on a developer laptop with OpenCV-only dependencies and scaling up to a GPU-accelerated production server with InsightFace and ArcFace, without changing a single line of client code.